

AGM - Algoritmo de Kruskal. (Goloso).

Ordena las aristas del grafo de forma creciente según su peso.

En cada paso elige la siguiente arista de mínimo peso que no forme un ciclo con las aristas ya elegidas.

El algoritmo frea cuando selecciona $n-1$ aristas.

Al comenzar el algoritmo, cuando todavía no se seleccionó ninguna arista, cada vértice forma una comp. conexa distinta.

En cada paso, une dos comp. conexas (hasta que queda un grafo conexo).

En cada iteración, si se elige la arista (u, v) , se une las comp. conexas de u y de v .

En cada paso, el bosque obtenido tiene una comp. conexa menos que el anterior.

$Kruskal(G) \rightarrow T = AGM \text{ de } G$

$x_T = \emptyset$

$cand = x$

como candidatas pone a todas

ordenar($cand$)

de menor a Mayor costo

$i = 1$

while $i \leq n-1$ do

$(u, w) = \min(cand)$

agarra la arista mínima

$cand = cand \setminus \{(u, w)\}$

y la saca.

if u y $w \notin$ a la misma comp. conexa de

$x_T = x_T \cup \{(u, w)\}$

$i = i + 1$

fin

fin

return $T = (V, x_T)$

AGM - Algoritmo de Prim (Goloso).

complejidad polinomial.

Empieza con un cjtto vacío. En cada paso elige una arista hasta formar un AGM.

Construye dos cjtos: uno de vértices V_T , inicializado con cualquier vértice. y otro de aristas X_T , que comienza vacío.

En cada iteración se agrega un elemento a cada cjtto.

Se selecciona la arista de menor costo entre las que tienen un extremo en V_T y el otro en $V \setminus V_T$.

Si hay empate se elige cualquiera.

Esta arista es agregada a X_T y el extremo V_T .

Cuando $V_T = V$ el algoritmo termina y las aristas de X_T definen un AGM de G .

Es una elección golosa: de un cjtto de aristas candidatas, elige a la mejor (la de menor costo).

$\text{Prim}(G) \rightarrow T = \text{AGM de } G$

$V_T = \{v\}$

cualquier vértice

$X_T = \emptyset$

$i = 1$

while $i \leq n-1$ do

$e = \text{argmin} \{l(e), e = (u, w), u \in V_T, w \in V \setminus V_T\}$

$X_T = X_T \cup \{e\}$

$V_T = V_T \cup \{w\}$

$i = i + 1$

fin

return $T = (V_T, X_T)$

Camino mínimo en digrafos.

Sea $G = (V, X)$ grafo y $l: X \rightarrow \mathbb{R}$ una función de longitud (peso) para las aristas de G .

La **longitud** de un camino C entre dos vértices v y u es la suma de las longitudes de las aristas del camino.

$$l(C) = \sum_{e \in C} l(e)$$

Un **camino mínimo** C^0 entre v y u es un camino entre v y u tq $l(C^0) = \min \{ l(C) \mid C \text{ es un camino entre } v \text{ y } u \}$

Puede haber varios caminos mínimos.

Problemas de camino mínimo:

1 - **Único origen-destino**: determinar un cam. min. entre dos vértices específicos.

2 - **Único origen - múltiples destinos**: determinar un cam. min. entre un vértice específico al resto de los vértices.

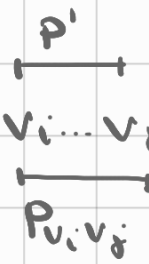
3 - **múltiples orígenes - múltiples destinos**: determinar un cam. min. entre todo par de vértices.

PROP. Subestructura óptima de un camino mínimo.

Dado un digrafo $G = (V, X)$ con una función de peso $l: X \rightarrow \mathbb{R}$. Sea $P: v_1, \dots, v_k$ un camino mínimo de v_1 a v_k . Entonces $\forall 1 \leq i \leq j \leq k$, $P_{v_i v_j}$ es un camino mínimo de v_i a v_j .

demo.

Por abs. P camino mínimo. $P: v_1 \dots v_i \dots v_j \dots v_k$



Supongamos que P no es mínimo. $\exists P' v_i \dots v_j$ con $l(P') < l(P_{v_i v_j})$

$$P'' = v_1 \dots v_i + P' + v_j \dots v_k$$

$$P'' = P_{v_1 v_i} + P' + P_{v_j v_k}$$

$$l(P'') = l(P_{v_1 v_i}) + \underbrace{l(P') + l(P_{v_j v_k})}_{< l(P_{v_i v_j})} < l(P) \quad \text{Abs! pues } P \text{ era el cam. min.}$$

Camino Maximin

Sea G un grafo, $c: E(G) \rightarrow \mathbb{R}$ una función de costos y $v, w \in V(G)$. Decimos que P es un **camino maximin** de v a w si se cumple que P maximiza $C_{\min}$.

$$C_{\min}(P) = \{c(vw) / vw \in E(P)\}$$

Queremos maximizar la "capacidad" que nuestro camino puede trasladar, y la capacidad de un camino está dada por la capacidad de su arista más pequeña.

Maximiza la mínima arista.

Camino Minimax

Sea G un grafo, $c: E(G) \rightarrow \mathbb{R}$ una función de costos y $v, w \in V(G)$. Decimos que P es un **camino minimax** de v a w si se cumple que P minimiza $C_{\max}$.

$$C_{\max}(P) = \{c(vw) / vw \in E(P)\}$$

Minimiza la máxima arista.

PROP. Camino AGM $\Rightarrow$ camino minimax

demo.

Por abs. supongamos que $\exists P$ camino Minimax entre los vértices a y z que no está completamente en el AGM T , siendo las aristas de T las azules.

Esto significa que hay una arista $N(\text{nueva}) = (s, t)$ en P que no está en T .

Sea Q el camino en T desde a hasta z .

Sea M (mayor) la arista de mayor peso en Q .



Hay 2 condiciones a considerar:

1 - $\text{peso}(M) > \text{peso}(N)$

En este caso, T no es un AGM pues $\exists T' = T - M + N$ / $\text{peso}(T') < \text{peso}(T)$. Abs!

2 - $\text{peso}(M) \leq \text{peso}(N)$

En este caso, podría eliminar N de P y agregar las aristas de M en su lugar, y seguiría siendo un camino minimax ya que no incluimos una arista con un peso mayor que el que ya estaba en ese camino antes.

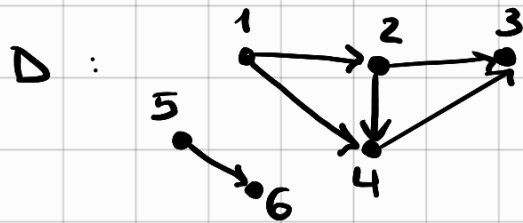
Por cada arista N en un camino minimax que no está en T, se puede realizar una sustitución de arista como se describe en el punto 2, creando así un camino minimax que estará completamente en T.

Orden topológico

Dado un digrafo D , un orden topológico de D es un ordenamiento $v_1, \dots, v_n$ de sus nodos que cumple que toda arista queda de la forma $v_i v_j$ con $i < j$ (en el ordenamiento).

Es decir, damos un orden a los nodos de tal forma que las aristas apuntan de izq. a der. (no hay aristas ~~para~~ atrás).

ejemplo.



Dos posibles órdenes topológicos :



- ¿Todo digrafo tiene orden topológico? No.
Si tiene ciclos, no tiene orden topológico.
- ¿Todo digrafo que admita orden topológico es un DAG? Sí.

↳ **Grafo acíclico dirigido**

Para saber si un grafo es un DAG, corra BFS o DFS.

↳ **Cosas que podemos resolver con DAGs:**

- I- # caminos que van de v a w .
- II- camino de menor/mayor costo de v a w .

Algoritmo de Kahn.

Se usa en DAGs.

Sirve para obtener el orden topológico de un DAG.

Si no es un DAG, detecta el ciclo.

- inicializo una lista vacía L que contendrá los nodos en orden topológico.
- inicializo un cto S con todos los nodos que no tengan aristas de entrada
- mientras S no esté vacío:
 - elimino un nodo n de S .
 - agrego n a L
 - para cada nodo m con una arista (n, m) :
 - elimino (n, m) de G
 - si m no tiene más aristas de entrada, agrego m a S
- si G aún tiene aristas, entonces tiene al menos un ciclo y no es posible un orden topológico
- de lo contrario, L contiene un orden topológico de los nodos.

Complejidad: $O(n+m)$ es lineal porque recorre los nodos y las aristas una sola vez.

Algoritmo de DIJKSTRA. (60/100)

Único origen - Grafo pesado.

asume que las longitudes de los arcos son **positivas**.

El grafo puede ser orientado o no orientado.

Construye un árbol de caminos mínimos.

comienza con el vértice v . En cada it, agrega al vértice más cercano a v , y así hasta agregar a todos.

lema dado un grafo orientado G con pesos no negativos en los arcos, al finalizar la it. k el algoritmo de Dijkstra determina el cam. min. desde el vértice v al resto de los vértices de G .

complejidad. los pasos computacionales son: encontrar el próximo vértice a agregar a S , y actualizar π .

- $O(m \log n)$ grafo poco denso
 - $O(n^2)$ grafo muy denso
- } $O(\min\{m \log n, n^2\})$

Algoritmo de Bellman-Ford. (P.D.)

Resuelve el problema de cau. min. con único origen (v) - Múltiples destinos, pero también cuando el digrafo puede tener arcos de longitud negativa.

Vamos a asumir que no hay ciclos de longitud negativa alcanzables desde v .

Para cada vértice u , en $\pi[u]$ va guardando una cota superior de la longitud de un cau. min. desde v a u (considera caminos con más cant. de arcos).

Logo de la it. k , el algoritmo encuentra todos los cau. min. desde v con, a lo sumo, k arcos.

El algoritmo se detiene cuando en una it. no es posible mejorar ningún camino. Como los caminos mínimos son simples, a lo sumo realiza $n-1$ iteraciones.

Detecta los ciclos de peso negativo. Si encuentra uno, $\nexists$ recorrido mínimo.



camino mínimo.

$\nexists$ recorrido mínimo.

complejidad. $O(nm)$

Algoritmos matriciales.

múltiples orígenes - múltiples destinos.

Sea $G=(V, X)$ un digrafo de n vértices y $l: X \rightarrow \mathbb{R}$ una función de peso para las aristas de G .

Definimos las siguientes matrices:

- $L \in \mathbb{R}^{n \times n}$, donde los elementos l_{ij} de L se definen como:

$$l_{ij} = \begin{cases} 0 & \text{si } i=j \\ l((v_i \rightarrow v_j)) & \text{si } (v_i \rightarrow v_j) \in X \\ \infty & \text{si } (v_i \rightarrow v_j) \notin X \end{cases}$$

- $D \in \mathbb{R}^{n \times n}$, donde los elementos d_{ij} de D se definen como:

$$d_{ij} = \begin{cases} \text{long. del cam. min. orientado de } v_i \text{ a } v_j & \text{si } \exists \text{ alguno} \\ \infty & \text{si no} \end{cases}$$

D = matriz de distancias de G .

Algoritmo de Floyd-Warshall (P.D.)

Determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

1_ Si $D^0 = L$ y calculamos D^1 como $d_{ij}^1 = \min(d_{ij}^0, d_{ii}^0 + d_{ij}^0)$
 d_{ij}^1 es la longitud de un cam. min. de v_i a v_j con nodo intermedio v_i o directo.

2_ Si calculamos D^k a partir de D^{k-1} como
 $d_{ij}^k = \min(d_{ij}^{k-1}, d_{ik}^{k-1} + d_{kj}^{k-1})$
 d_{ij}^k es la longitud de los cam. min. de v_i a v_j cuyos nodos intermedios son elementos de $V_k = \{v_1, \dots, v_k\}$, si $\nexists$ ciclo de longitud negativa con todos sus vértices en V_k .

3_ $D = D^n$

lema. Si G tiene un ciclo de longitud negativa $\Rightarrow d_{ii}^n < 0$ para algún i .

- Se implementa bottom-up
- me tengo que guardar la matriz de distancias
- devuelve una matriz.
- en cada paso k se tiene acceso a todos los caminos mínimos dentro del cjtto $\{1, \dots, k\}$
- en la diagonal de la matriz voy a tener todos ceros
- sirve para detectar ciclos de longitud negativa.

Subproblema de PD: la distancia mínima entre i y j usando solo los primeros k vértices como intermedios.

Complejidad temporal: $O(n^3)$ pues realiza 3 bucles anidados:

```
for k in range(n):           // nodo intermedio
    for i in range(n):       // nodo origen
        for j in range(n):   // nodo destino
            dist[i][j] = min(dist[i][j], dist[i][k] +
                               dist[k][j])
```

Complejidad espacial: $O(n^2)$ pues usa una matriz de distancias de tamaño $n \times n$.

invariante: después de la it. k , la entrada $d[i][j]$ contiene la longitud del camino mínimo entre los vértices i y j cuyos vértices intermedios (si los hay) pertenecen únicamente al cjtto $\{1, 2, \dots, k\}$

Algoritmo de Dantzig. (P.D.)

Determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin ciclos de longitud negativa.

Al finalizar la it. $k-1$, Dantzig genera una matriz de $k \times k$ de caminos mínimos en el subgrafo inducido por los vértices $\{v_1, \dots, v_k\}$

Calcula la matriz L^{k+1} a partir de la matriz L^k para $1 \leq i, j \leq k$ como:

$$\triangleright L_{j,k+1}^{k+1} = \min_{1 \leq j \leq k} (L_{i,j}^k + L_{j,k+1}^k)$$

$$\triangleright L_{k+1,i}^{k+1} = \min_{1 \leq j \leq k} (L_{k+1,j}^k + L_{j,i}^k)$$

$$\triangleright L_{i,j}^{k+1} = \min (L_{i,j}^k, L_{i,k+1}^k + L_{k+1,j}^k)$$

- sirve para detectar ciclos de longitud negativa.
- misma complejidad que Floyd-Warshall: $O(n^3)$

invariante: después de la it. k , todos los caminos mínimos entre los vértices i y j que sólo utilizan vértices con índices menores o iguales a k , ya fueron correctamente determinados en la matriz D .

Floyd-Warshall

VS

Dantzig

¿Qué asegura el invariante?

Distancia mínima usando
intermedios en $\{1, \dots, k\}$

Distancia mínima entre
vértices del subgrafo $\{1, \dots, k\}$

¿Qué cambia en cada paso?

Se habilita un vértice
como intermedio posible
para todo el grafo

Se incorpora un nuevo
vértice al subgrafo "ya
resuelto"

¿Qué obtengo al final?

Todas las distancias
mínimas entre todos
los pares

Todas las distancias
mínimas entre todos
los pares

Algoritmo de Johnson.

Calcula camino mínimo todos a todos en grafos sin ciclos negativos.

Admite aristas negativas.

Le das un grafo y te da la matriz de caminos min.

1º) hace un Bellman-Ford para conseguir un grafo sin aristas negativas.

2º) corre Dijkstra n veces.

$$\text{Complejidad} = O(\underbrace{nm}_{\text{Bellman-Ford}} + \underbrace{nm \log n}_{\text{Dijkstra}}) = O(nm \log n)$$

Bellman-Ford

Dijkstra

Si no tengo aristas negativas:

↳ Johnson corre Dijkstra n veces en $O(nm \log n)$

¿Cuándo conviene usar Johnson o Floyd-Warshall?

Si el grafo es completo, o sea que la cant. de aristas es n^2 , entonces Johnson me quedaría

$$O(nm \log n) = O(n \cdot n^2 \log n) = O(n^3 \log n)$$

y como Floyd-Warshall cuesta $O(n^3)$, esta complej. es mejor.

En caso contrario, es un grafo raro, donde $m = O(n)$ tendríamos $O(n^3)$ con Floyd vs $O(n^2 \log n)$ con Johnson o sea que conviene Johnson.

Funcionamiento de Johnson :

1. Se añade un vértice y aristas desde este nuevo nodo a todos los demás vértices, todos con peso cero.

Este nuevo vértice se convierte en el origen de todos los caminos de mi nuevo grafo.

El objetivo es transformar el grafo original para eliminar los pesos negativos.

2. Aplicamos Bellman-Ford para encontrar los caminos mínimos desde el nuevo vértice a todos los demás.

3. Se reasignan los pesos originales de las aristas del grafo para eliminar las adiciones ficticias que se hicieron.

4. Aplicamos Dijkstra desde cada vértice para determinar los caminos mínimos entre cada par de vértices en el grafo original.